МАКСИМУМ
В ЦИФРЕ

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

Спасти рядового Райана,
или
как извлечь одну таблицу PostgreSQL из терабайтного бэкапа

Михаил Торговкин

Приветствие

Доклад посвящён низкоуровневому восстановлению одной таблицы PostgreSQL из огромного бэкапа с использованием знаний о файловой структуре и утилиты pg_filedump.

Вопросы, на которые ответим:

- Как отделить пользовательские данные от системных?
- Как узнать имена физических файлов таблицы и её TOAST-объектов?
- Как скопировать только нужные файлы из резервной копии?
- Как применить эти знания в критической ситуации?

Приготовьтесь к погружению во внутренности PostgreSQL!

Зачем создавать свой *tablespace* по умолчанию?

При проектировании кластера PostgreSQL можно создать отдельный tablespace и сделать его tablespace по умолчанию для пользовательских данных.

Преимущества:

- ✓ Данные пользовательских таблиц не смешиваются с системными файлами (каталог **pg_default**).
- ✓ Упрощается восстановление после сбоев: можно манипулировать отдельными tablespace.
- ✓ Снижается нагрузка на файловую систему при чтении каталога с большим количеством файлов.

Недостаток:

- ✕ При использовании **pg_basebackup** без архивации WAL необходимо явно перечислять все дополнительные **tablespace** в параметре **--tablespace**.

Создание базового бэкапа с *pg_basebackup*

создания физической копии кластера используйте утилиту **pg_basebackup**. Рекомендуется параметр **-c fast**, чтобы не ждать контрольную точку.

```
pg_basebackup \
  -h localhost \
  -D "$BACKUP_DIR" \
  -X stream \
  -P -v \
  -c fast \
  -U postgres
```

План восстановления: основные шаги

Последовательность действий.

Шаг 1.
Подготовка целевой таблицы

- Создание таблицы **table_restore** в отдельном **tablespace**.
- Таблица создаётся без индексов для ускорения загрузки данных.

Шаг 2.
Получение метаданных из pg_class

- Запрос к системному каталогу для получения OID таблицы, relfilenode и OID TOAST-таблицы.
- Понимание связи между логическими и физическими именами.

Шаг 3.
Определение TOAST-таблиц и их индексов

- Поиск всех объектов, связанных с основной таблицей (TOAST-таблица и её индекс).
- Запись имён файлов (*relfilenode*) для дальнейшего копирования.

Шаг 4.
Использование pg_filedump для прямого чтения системного каталога

- Получение данных напрямую из файла 1259 (*pg_class*).
- Декодирование нужных колонок с помощью опции -D.

Шаг 5.
Локализация файлов данных

- Идентификация фактических имён файлов на диске по номерам **relfilenode**.
- Подготовка к копированию или восстановлению соответствующих файлов.

Результат: мы получаем полный список файлов, которые необходимо скопировать из резервной копии или другого источника для восстановления конкретной таблицы и её TOAST-данных.

Шаг 1. Подготовка к восстановлению

Создайте структуру таблицы, в которую будут восстановлены данные.

Используйте отдельный **tablespace** для наглядности при работе с файлами.

sql

```
CREATE TABLE table_restore (
    id bigint,
    date timestamp,
    code varchar(10),
    description text,
    opt text
) TABLESPACE table_space_restore;
```

Важно:
индексы и ограничения можно создать позже, после восстановления данных, чтобы ускорить процесс.

Шаг 2. Получение информации о файлах таблицы из **pg_class**

Запросите системный каталог **pg_class**, чтобы узнать OID таблицы, имя файла (relfilenode) и OID связанной TOAST-таблицы.

sql

```
SELECT
    oid,
    oid::regclass AS rel_name,
    relfilenode,      -- имя файла на диске
    reltoastrelid     -- OID TOAST-таблицы
FROM pg_class
WHERE oid = 'table_restore'::regclass;
```

Результат (пример):

oid	rel_name	relfilenode	reltoastrelid
2136093	table_restore	2136093	2136096

Обратите внимание:
relfilenode может отличаться от OID после операций TRUNCATE или VACUUM FULL.

Шаг 3. Получение информации о TOAST-таблицах

TOAST-таблицы хранят длинные значения.
Их имена содержат OID основной таблицы.

sql

```
SELECT
    oid,
    oid::regclass AS rel_name,
    relfilenode,      -- имя файла на диске
    reltoastrelid     -- обычно 0 для TOAST-таблиц
FROM pg_class
WHERE oid::regclass::text LIKE
'%pg_toast_2136093%';
```

Результат (пример):

oid	rel_name	relfilenode	reltoastrelid
2136096	pg_toast_2136093	2136096	0
2136097	pg_toast_2136093 _index	2136097	0

Имена TOAST-таблиц и их индексов всегда начинаются с pg_toast_ и заканчиваются OID основной таблицы.

Шаг 4. pg_filedump — наше всё! 😊

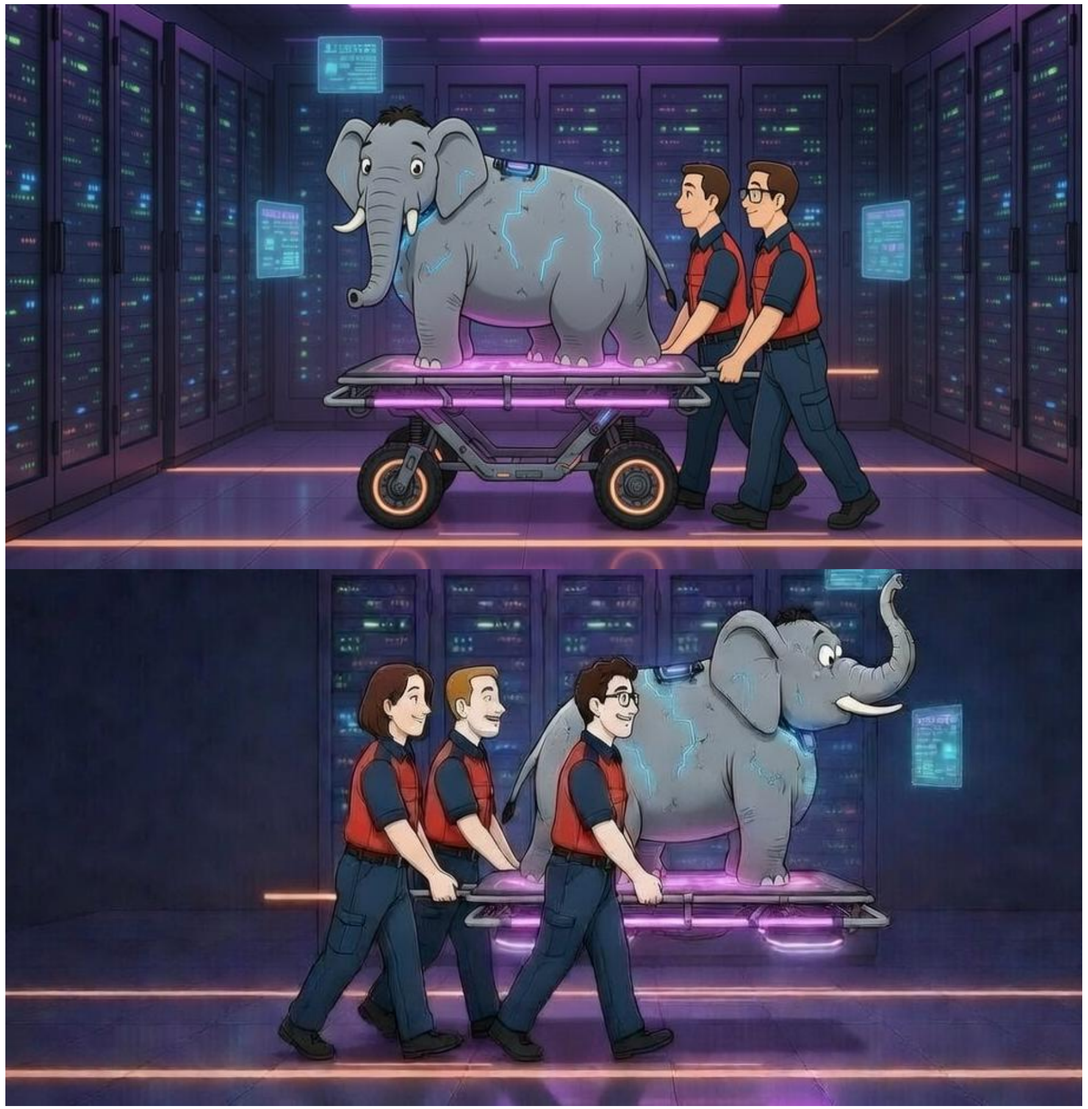
pg_filedump — утилита для низкоуровневого просмотра содержимого файлов данных PostgreSQL.

Позволяет читать системные каталоги напрямую, даже если кластер не запущен.

- Полезна при восстановлении, анализе повреждений, изучении внутреннего устройства.
- Выводит данные в сыром или структурированном виде *(с опцией -D)*.

Установка (на примере *Debian/Ubuntu*):

```
bash
apt-get install pg-filedump
```



Шаг 4. Получение имён файлов таблицы через **pg_filedump**

Файл системного каталога **pg_class** имеет номер **1259**.

Прочитаем его, декодируя нужные поля для **table_small**.

```
pg_filedump -o -D
oid,name,oid,oid,oid,oid,oid,oid,oid,
int,real,int,int,oid 1259 \
  | sed -n 's/.*Partial data: *//p' \
  | awk 'BEGIN {FS=OFS="\t"} { $8 = "\033[36m"
$8 "\033[0m"; print }' \
  | grep -w table_small
```

Результат (только интересующие колонки):

oid	rel_name	relfilenode	reltoastrelid
1016467	table_small	2036089	1016472

Пояснение:

- oid = 1016467 идентификатор таблицы
- (8 поле) relfilenode = 2036089 физическое имя файла с данными таблицы
- (14 поле) reltoastrelid = 1016472 — OID TOAST-таблицы, связанной с данной таблицей

Шаг 5. Получение имён файлов TOAST через **pg_filedump**

Теперь, зная OID TOAST-таблицы (1016472), найдём соответствующие файлы TOAST и его индекса.

```
pg_filedump -o -D
oid,name,oid,oid,oid,oid,oid,oid,oid,
int,real,int,int,oid 1259 \
  | sed -n 's/.*Partial data: *//p' \
  | awk 'BEGIN {FS=OFS="\t"} { $8 = "\033[36m"
$8 "\033[0m"; NF=8; print }' \
  | grep pg_toast_1016467
```

Результат (пример):

oid	rel_name	relfilenode
1016472	pg_toast_1016467	2036090
1016473	pg_toast_1016467_index	2036091

- **2036090** — номер файла на диске для TOAST-таблицы
- **2036091** — номер файла на диске для индекса TOAST

Заключение

- Создание собственного *tablespace* по умолчанию упрощает администрирование и восстановление.
- Знание имён файлов данных позволяет гибко управлять восстановлением, особенно при использовании низкоуровневых инструментов.
- Утилита **pg_filedump** — мощное средство для исследования файлов *PostgreSQL*.
- Всегда проверяйте соответствие *relfilenode* и *OID* после операций, меняющих физическое представление таблиц.

Копирование файлов. Интерактив

Теперь, когда мы знаем имена файлов исходной таблицы **(2036089, 2036090, 2036091)** из резервной копии, а также новые имена *relfilenode* для созданной таблицы **table_restore**.

Для **table_restore** мы получили:

- *relfilenode* таблицы = **2136093**
- *relfilenode TOAST*-таблицы = **2136096**
- *relfilenode* индекса TOAST = **2136097**

Тогда порядок действий:

1. Если это прод, желательно остановить целевой кластер PostgreSQL.
2. Скопировать файлы с исходными именами из бэкапа:
   - 2036089 (данные таблицы)
   - 2036090 (TOAST-таблица)
   - 2036091 (индекс TOAST)
3. Переименовать их в соответствии с новыми именами relfilenode:
   - 2036089 → 2136093
   - 2036090 → 2136096
   - 2036091 → 2136097
4. Поместить переименованные файлы в каталог tablespace новой таблицы:
   - $PGDATA/pg_tblspc/<tablespace_oid>/<database_oid>/
5. Убедиться, что права доступа совпадают (обычно *0700* для каталогов, *0600* для файлов).

МАКСИМУМ
В ЦИФРЕ

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

Ваши вопросы?